MILITARY INSTITUTE OF SCIENCE AND TECHNOLOGY

ResultLang: Design, Implementation, and Evaluation of a Domain-Specific Result Processing Compiler

Compiler Design Project Report

COURSE	CSE 303 Compiler Design
PREPARED BY	Md. Mahdi Israk Mahir (202414016) Rafsan Jony Asif (202414019)
BASIS	Complete ResultLang project repository
DATE	18 September 2026

Academic Project Submission

Contents

1. Introduction
2. Methodology
3. Results and Outcomes
4. Result Representation and Bit Width
5. Connecting a Mobile Device on the Same IP Network
6. Critical Analysis
7. Conclusion
8. References
9. Appendix A: Project File Inventory
10. Appendix B: Language Feature Summary
11. Appendix D: Validation Record
12. Appendix K: Implementation Glossary and Reproducibility Protocol

Abstract

ResultLang is a compact domain-specific language for expressing student marks, averages, grade decisions, and output. This report examines its complete compiler pipeline, including lexical analysis, parsing, abstract syntax trees, semantic checking, symbol management, three-address code, C generation, executable construction, testing, limitations, and future improvements. Twenty automated cases validate successful compilation and error rejection.

1. Introduction

1.1 Background

Compiler construction is often taught through isolated algorithms: a lexer recognizes tokens, a parser builds structure, semantic analysis checks meaning, and a back end emits another representation. A small working language makes those ideas observable as one connected system. ResultLang applies that model to a deliberately narrow educational problem: representing a student record, calculating marks and averages, choosing a grade, and displaying the result. The language is not intended to replace a general-purpose programming language. Its value is that every source statement can be traced through a complete implementation.

The project accepts ResultLang source files with identifiers, strings, numbers, Boolean literals, assignments, arithmetic expressions, comparisons, conditional statements, and a show operation. The verified implementation translates a source file into an abstract syntax tree, writes a symbol table and intermediate three-address code, generates C, and relies on GCC to produce the final executable. This staged structure reflects the compiler pipeline described in contemporary compiler engineering literature, where representation changes make analysis and code generation easier to reason about (Cooper & Torczon, 2022).

The repository also contains Flex and Bison descriptions. Those files are valuable as a declarative account of the intended lexical and grammatical design. However, the project README records that the final verified environment did not contain Flex and Bison, so the executable build uses the self-contained implementation in `src/main.c`. This distinction matters for reproducibility: a report must describe what was actually executed and separately identify artifacts that document an alternative implementation route.

1.2 Objectives

The project objectives were to design a small result-oriented language; implement the principal front-end stages; infer and record simple static types; reject malformed or unsafe

programs with line-numbered diagnostics; expose intermediate compiler artifacts for inspection; generate portable C source; compile that source with GCC; and demonstrate the end-to-end behavior using valid and invalid examples. The report has an additional objective: to evaluate not only whether the implementation works, but also where its design is intentionally limited and where the implementation should be strengthened before being treated as a production compiler.

More specifically, the language should support a student name, subject marks, computed totals, averages, comparison-based grade selection, and visible output. The compiler should preserve arithmetic precedence, represent nested conditionals, distinguish number, string, and Boolean values, detect undefined identifiers, prevent incompatible reassignment, reject invalid comparisons, identify string arithmetic, and report division by zero when it is statically visible. The build process should be simple enough for a learner to execute on Windows using GCC. Finally, an automated test suite should cover both success and failure paths instead of only demonstrating one successful example.

2. Methodology

2.1 Repository inspection and evidence collection

The methodology began with a source-level inspection of the repository. The README established the language scope and commands. The source directory was then read in pipeline order: the lexer specification, parser specification, AST module, semantic module, symbol table module, TAC module, C code generator, and the self-contained compiler driver. Examples in the valid and invalid directories were examined as executable specifications. Generated AST, symbol table, TAC, and C files were treated as implementation evidence rather than as assumptions about intended behavior. Build scripts and the Python test runner were read to identify the exact commands used by the project.

This approach separates three kinds of evidence. First, source evidence establishes what the compiler is programmed to do. Second, generated artifacts demonstrate how one concrete program passes through the stages. Third, test evidence establishes which behaviors were exercised automatically. The distinction is important because a compiler may contain an intended grammar that differs from its deployed driver, or may implement a diagnostic without testing it. Findings in this report therefore use the verified executable path as the primary authority and use the Flex/Bison files as design documentation.

2.2 Front-end method

The deployed driver reads the complete source file into memory, tokenizes it, parses the token stream with recursive-descent functions, performs semantic analysis over the

resulting tree, and only then writes compiler outputs. Lexical analysis tracks source lines and recognizes identifiers, numeric literals, strings, keywords, operators, delimiters, and comments. It reports unknown characters, malformed numbers with multiple decimal points, and unterminated strings. The parser implements primary expressions, multiplication and division, addition and subtraction, and one comparison level. This ordering gives multiplication higher precedence than addition and makes comparison lower precedence than arithmetic. Parentheses recursively invoke expression parsing.

Statements include assignment, show, and if/else. The parser accepts both block forms using braces and a single statement as a branch. The grammar requires an else branch. Assignments have a presentation-oriented shorthand: an identifier followed directly by an expression is accepted in addition to an identifier followed by an equals sign. This is why the sample can write `student "Rahim";`. The AST stores source line numbers and links statements through a list. Binary nodes retain their operator and children, while conditional nodes retain condition, then, and else subtrees.

2.3 Semantic method

Semantic analysis traverses expressions recursively and assigns one of four types: unknown, number, string, or bool. Literals are typed immediately. An identifier must already exist in the symbol table. Arithmetic requires two numeric operands and produces a number. Comparisons require matching known operand types and produce a Boolean result. An if condition must therefore be a Boolean comparison. When a variable is first assigned, its inferred type is recorded with its first source line. A later assignment with a different type produces a type mismatch diagnostic. This is a small but coherent form of static typing.

The semantic pass also checks division by a literal zero. The generated C contains a runtime helper, `rl_safe_div`, so the same operation remains guarded when the divisor is not a compile-time constant. The analysis intentionally does not attempt full constant propagation, path-sensitive analysis, or control-flow typing. It checks statements in source order and recursively checks both branches of a conditional. That makes the implementation easy to follow, but it also means that sophisticated definite-assignment and branch-join rules are outside the current language model.

2.4 Intermediate and back-end method

After semantic success, the compiler emits four artifacts. The AST is a readable tree used to inspect syntactic structure and inferred binary-expression types. The CSV symbol table records names, types, and first declaration lines. TAC lowers binary expressions into temporaries and lowers conditionals into conditional jumps and labels. The C generator

declares variables according to their inferred type, translates expressions recursively, emits formatted output based on expression type, and translates if/else into C control flow. The generated C is then compiled by GCC in the scripts and test suite.

The evaluation method used `scripts/test_all.py`. It first compiles `src/main.c` with C11, warnings enabled, and optimization. Each of ten valid examples is compiled by ResultLang, compiled again as generated C, executed, and compared with an exact expected-output list. Each of ten invalid examples must produce a non-zero compiler status, contain the expected diagnostic phase, and avoid creating `generated.c`. This is a useful pipeline test because it catches errors that would be invisible in a parser-only test.

3. Results and Outcomes

3.1 Functional outcome

The complete automated run executed 20 tests and passed all 20. The ten valid tests cover assignment and output, arithmetic precedence, total and average calculation, passing and failing grades, a boundary value at 80, comparisons, strings and booleans, nested expressions, and a full result demonstration. Each valid program passed through ResultLang compilation, generated-C compilation, and executable execution. The test runner reported: “Tests run: 20 passed: 20 failed: 0” and “ALL TESTS PASSED.” This result provides strong evidence that the checked examples are stable across the complete implemented pipeline.

The ten invalid tests cover an unknown character, missing semicolon, invalid expression, missing else, undefined variable, type mismatch, string arithmetic, division by zero, malformed string, and invalid comparison. They were classified into lexical, syntax, and semantic categories. The compiler rejected each case before generated C was accepted as a successful output. This is particularly significant because error handling is part of the language contract: a compiler that produces plausible output for an invalid source program would be worse than one that stops with a precise diagnostic.

3.2 Concrete artifact outcome

The generated artifact set makes the compilation stages inspectable. For the full result demonstration, the AST begins with Program and StatementList nodes, then records assignments for the student and marks, binary nodes for total and average, and a nested IfElse structure for grade thresholds. The comparison `average >= 80` is labelled bool, while the total and average arithmetic nodes are labelled number. The tree therefore confirms both parsing structure and semantic annotation.

The symbol table records student as string, DBMS as number, Compiler as number, MMA as number, total as number, average as number, and grade as string, together with first source lines. This is enough information for the C generator to emit a `const char *` declaration for string values and `double` declarations for numeric values. Boolean values are emitted as integer storage and displayed as the words true or false.

The TAC output demonstrates lowering. For example, the total expression becomes a sequence such as `t1 = DBMS + Compiler`, `t2 = t1 + MMA`, and `total = t2`. Conditions become temporaries followed by `ifFalse` jumps. Nested grade selection is represented through labels L1 through L12, unconditional jumps, and fall-through blocks. TAC is not executed directly, but it is a useful audit surface: it reveals evaluation order, temporary creation, and branch structure before C generation.

3.3 Usability outcome

The project provides batch scripts for building the compiler, compiling a selected file, running the demo, and entering a program interactively. The interactive runner reads lines until END, saves them as a ResultLang source file, rebuilds the compiler, generates C, builds an executable, and prints its result. This workflow reduces the distance between language design and demonstration. A learner can inspect both the source-level result and the intermediate files without needing a separate IDE plugin.

The use of C as a target also keeps the back end understandable. The output is ordinary C with standard input-independent execution, formatted output, and a small division helper. GCC supplies the final native compilation step. The trade-off is that the generated C inherits restrictions from the target language and from the code generator. For example, identifiers are copied into C declarations without a dedicated target-language namespace policy, and string values are represented as fixed generated literals rather than a richer runtime value system.

3.4 Result representation and bit width

The verified desktop implementation represents numeric ResultLang values as C `double`. On the tested 64-bit Windows GCC environment, this is the IEEE-754 binary64 format and occupies 64 bits (8 bytes), so marks, totals, and averages use that representation. Boolean values are emitted as C `int`; the tested environment uses 32 bits (4 bytes). The final grade and other strings are emitted as `const char *`. On a 64-bit desktop, the pointer occupies 64 bits, while the characters occupy a variable number of bytes: an ASCII string uses one byte per character plus its terminating null byte.

These sizes describe the generated desktop executable, not a separate mobile executable. The mobile device in this project is a client that views the report and generated artifacts over the same local network. Therefore, it does not change the compiler's result representation. A future native mobile port should verify the target ABI with `sizeof(double)`, `sizeof(int)`, and `sizeof(char *)`; a 64-bit ARM build will commonly use the same 64-bit, 32-bit, and 64-bit sizes respectively, but that is an observed target property rather than a language guarantee.

3.5 Connecting a mobile device on the same IP network

To connect a phone or tablet for local inspection, both devices were placed on the same Wi-Fi network. The computer's local IPv4 address was found with `ipconfig`. From the ResultLang repository root, the project files were shared with Python's built-in server using `python -m http.server 8000 --bind 0.0.0.0`. On the mobile device, the address `http://COMPUTER_IP:8000/` was opened, with `COMPUTER_IP` replaced by the computer address, for example `192.168.1.25`. Windows Firewall was allowed to accept Python connections on the private network when prompted.

This setup lets the mobile browser inspect `ResultLang_Report.html`, the README, examples, and generated AST, TAC, symbol-table, and C files. It does not run the Windows executable on the phone: the compiler and generated program continue to run on the computer. Both devices must remain on the same LAN and the Python server must remain active. This is a local network demonstration, not public internet deployment.

4. Critical Analysis

4.1 Strengths

The strongest feature is end-to-end completeness relative to the project scope. ResultLang does not stop at token recognition or parse-tree construction. It carries typed structure through analysis, intermediate representation, source generation, native compilation, and execution. That makes the project pedagogically valuable because a student can see how each compiler stage contributes to an observable result.

The second strength is the explicit separation of failure phases. Diagnostics are labelled lexical, syntax, or semantic and include source line numbers. The test suite checks these labels. The invalid examples are not merely malformed random inputs; they represent distinct contracts, including undefined names, type errors, invalid operators, and a safety check for division by zero.

The third strength is the inspectable artifact design. AST, CSV, TAC, and C outputs allow a reviewer to compare source intent with every intermediate form. This supports debugging and assessment. It also aligns with modern engineering practice in which reproducible build and validation artifacts are more useful than an undocumented executable claim (NIST, 2022).

4.2 Limitations and risks

The implementation is intentionally bounded. It stores tokens, symbols, and AST nodes in fixed-size arrays in the self-contained driver. Limits such as 8,192 tokens, 4,096 nodes, 512 symbols, and 256-character token text are reasonable for classroom examples but should be reported as capacity constraints. The AST and symbol table code in the modular files use dynamic AST allocation and named limits, while the verified driver uses a separate compact representation. Maintaining two implementations increases the risk of behavioral drift.

The parser's assignment rule is permissive: after an identifier, it consumes an equals sign if present and then parses an expression. This intentionally supports the shorthand syntax, but it also means the grammar policy should be documented carefully. A future implementation should make the two forms explicit in both the parser and its tests. The Flex/Bison grammar already represents this idea with `optional_assign`, but the build does not currently generate the deployed executable from those files.

Type checking is useful but not complete. Comparisons permit equality or ordering between any two matching types, even though ordering strings may be questionable for the intended domain. Arithmetic errors are reported, but the expression is still assigned number type, which is a recovery choice rather than a formal type-theoretic result. The compiler checks division by a literal numeric zero but cannot prove that a variable divisor is nonzero. The runtime helper closes that gap for generated C, although runtime error reporting could be made more consistent with compiler diagnostics.

The generated C back end also deserves hardening. It escapes quotes and backslashes in string literals, which is a good baseline, but the language should define escape semantics more precisely. Generated identifiers should be validated against reserved C keywords and target naming rules. The output currently uses a simple C main function and no runtime representation for strings beyond pointers. If ResultLang gained more types or operations, a tagged value representation or a stronger target abstraction would become preferable.

The test suite is broad for a small project but remains example-driven. It does not independently unit-test every lexer rule, parser precedence combination, AST memory path, maximum-capacity boundary, generated identifier collision, or escaped string. It also

does not run sanitizers, fuzzing, mutation testing, or cross-platform compiler comparisons. The 20 passing tests should therefore be interpreted as strong evidence for the included scenarios, not proof that all possible ResultLang programs are correct.

4.3 Comparison with compiler engineering practice

ResultLang follows the recognizable front-end sequence of scanning, parsing, semantic analysis, and intermediate representation. Its recursive-descent parser makes precedence visible in function boundaries, while the Bison file expresses precedence declaratively. Its symbol table records type information and source location, and its TAC makes control flow explicit before target emission. These are appropriate educational choices because they expose concepts that are often hidden by industrial compiler frameworks (Cooper & Torczon, 2022).

Compared with industrial compilers, the project omits recovery strategies, separate compilation, optimization, source maps, richer diagnostics, runtime libraries, and formal intermediate representation invariants. Those omissions are not defects when measured against the stated domain. They do define the boundary of the result: ResultLang is a demonstrator and teaching compiler with a complete path, not a general language implementation.

5. Conclusion

ResultLang successfully demonstrates a complete small-language compiler for academic result processing. Its verified driver recognizes source text, builds typed structure, rejects the tested error classes, emits AST, symbol table, TAC, and C artifacts, and produces correct executable output for ten valid examples. The 20-case suite provides reproducible evidence of this outcome. The project is strongest as a transparent teaching system. Its principal risks are the coexistence of two front-end implementations, fixed capacity limits, incomplete type-flow analysis, and limited back-end hardening. Future work should unify the build path, expand tests, improve diagnostics, define escape and identifier rules, and add automated sanitizer and boundary testing. Within its intended scope, however, the implementation is coherent, demonstrable, and critically assessable.

References

Cooper, K. D., & Torczon, L. (2022). *Engineering a compiler* (3rd ed.). Morgan Kaufmann.

Free Software Foundation. (2024). *Using the GNU compiler collection (GCC)*. https://gcc.gnu.org/onlinedocs/gcc/

Free Software Foundation. (2024). *The GNU Bison manual*. https://www.gnu.org/software/bison/manual/

GNU Flex Project. (2024). *Flex: The fast lexical analyzer generator manual*. https://westes.github.io/flex/manual/

International Organization for Standardization. (2024). *ISO/IEC 9899:2024: Programming languages - C*. https://www.iso.org/standard/82075.html

National Institute of Standards and Technology. (2022). *Secure software development framework (SSDF) version 1.1: Recommendations for mitigating the risk of software vulnerabilities* (SP 800-218). https://doi.org/10.6028/NIST.SP.800-218

OWASP Foundation. (2021). *OWASP Top 10: The ten most critical web application security risks*. https://owasp.org/Top10/

Python Software Foundation. (2024). *subprocess - Subprocess management*. https://docs.python.org/3/library/subprocess.html

SEI CERT Coordination Center. (2024). *SEI CERT C coding standard*. Carnegie Mellon University. https://wiki.sei.cmu.edu/confluence/display/c

W3C. (2023). *HTML standard*. https://html.spec.whatwg.org/

MITRE. (2024). *Common weakness enumeration (CWE)*. https://cwe.mitre.org/

Appendix A: Project File Inventory

Path	Role	Evidence or contribution
README.md	Project guide	Scope, syntax, prerequisites, commands, outputs, limitations, and team information.
src/main.c	Verified compiler driver	Self-contained lexer, recursive-descent parser, AST, semantics, symbol table, TAC, C generator, and command-line entry point.
src/lexer.l	Declarative lexer specification	Flex rules for keywords, identifiers, literals, operators, comments, and bad tokens.
src/parser.y	Declarative grammar specification	Bison grammar for statements, optional assignment, expressions, precedence, and if/else.
src/ast.c, src/ast.h	Modular AST implementation	Dynamic AST construction, printing, list append, and recursive freeing.
src/semantic.c, src/semantic.h	Modular semantic layer	Type inference, identifier lookup, operator validation, and semantic diagnostics.
src/symbol_table.c, src/symbol_table.h	Symbol management	Typed entries, first source line, duplicate type checking, and CSV-style dump.
src/tac.c, src/tac.h	Intermediate representation	Temporary values, labels, branch lowering, and show calls.
src/codegen.c, src/codegen.h	C back end	C declarations, expressions, output formatting, if/else translation, and safe division.
examples/valid	Positive examples	Ten expected-success programs for language and pipeline behavior.

examples/invalid	Negative examples	Ten expected-rejection programs for lexical, syntax, and semantic diagnostics.
scripts/test_all.py	Automated evaluation	Builds, compiles, executes, compares outputs, validates errors, and writes summaries.
scripts/run_user_input.py	Interactive workflow	Reads source until END and runs the complete compile-to-executable path.
scripts/build.bat	Compiler build script	Builds the self-contained driver with GCC C11 and warnings.
scripts/compile_file.bat	File compilation script	Compiles a ResultLang source file and then builds generated C.
generated/	Compiler evidence	AST, symbol table, TAC, generated C, and executable outputs from a sample run.
tests/TEST_PLAN.md	Test documentation	Summarizes the ten valid and ten invalid pipeline cases.

Appendix B: Language Feature Summary

Feature	Example	Static result
String assignment	`student "Rahim";`	student becomes string.
Numeric assignment	`math = 85;`	math becomes number.
Arithmetic	`average = total / 3;`	number expression with precedence.
Comparison	`average >= 80`	Boolean expression.
Conditional	`if (condition) statement else statement`	Two control-flow branches.
Output	`show(grade);`	Typed formatted output.
Comments	`# comment` or `// comment`	Ignored by the lexer.

Appendix C: Representative Source and Generated Forms

C.1 Source

```
student "Rahim";
math = 85;
english = 78;
science = 90;
total = math + english + science;
average = total / 3;
if (average >= 80)
  grade = "A+";
else
  grade = "B";
show(grade);
```

C.2 Representative TAC

```
student = "Rahim"
math = 85
english = 78
science = 90
t1 = math + english
t2 = t1 + science
total = t2
t3 = total / 3
average = t3
t4 = average >= 80
ifFalse t4 goto L1
grade = "A+"
goto L2
L1:
grade = "B"
L2:
param grade
call show
```

C.3 Representative generated C

```
#include <stdio.h>
#include <stdlib.h>

static double rl_safe_div(double a, double b) {
    if (b == 0) {
        fprintf(stderr, "runtime error: division by zero\n");
        exit(2);
    }
    return a / b;
}

int main(void) {
    const char *student = "";
    double math = 0;
    double english = 0;
    double science = 0;
    double total = 0;
    double average = 0;
    const char *grade = "";
    student = "Rahim";
```

```
    math = 85;
    english = 78;
    science = 90;
    total = ((math + english) + science);
    average = rl_safe_div(total, 3);
    if (average >= 80) {
        grade = "A+";
    } else {
        grade = "B";
    }
    printf("%s\n", grade);
    return 0;
}
```

Appendix D: Validation Record

The validation command was `python scripts\test_all.py`. It rebuilt the compiler with GCC, ran ten valid examples through ResultLang, GCC, and the executable, ran ten invalid examples against diagnostic expectations, and wrote machine-readable and Markdown summaries under the generated test output directory. The observed result was 20 tests run, 20 passed, and 0 failed.

Category	Count	Result
Valid end-to-end pipeline	10	10 passed
Invalid lexical, syntax, and semantic rejection	10	10 passed
Total	20	20 passed, 0 failed

Appendix E: Recommended Future Work

First, make one front end authoritative. Either generate the deployed compiler from Flex and Bison or clearly mark those files as historical design artifacts. Second, add unit tests around token boundaries, decimal errors, escaped strings, precedence, optional assignment, and nested branch parsing. Third, add tests for fixed capacities and compile the driver under AddressSanitizer and UndefinedBehaviorSanitizer. Fourth, define a formal ResultLang grammar and type table so implementation choices are not inferred only from code. Fifth, improve diagnostics by reporting source excerpts, expected token sets, and related locations for incompatible assignments. Sixth, validate identifiers before copying them into generated C and reserve a target-language escape strategy. Seventh, consider a richer intermediate representation with explicit instruction records rather than writing TAC directly to a file. Finally, introduce reproducible build metadata that records GCC version, operating system, compiler flags, test timestamp, and source revision. These changes would preserve the project's transparency while increasing confidence in its portability and maintainability.

Appendix F: Extended Technical Analysis

F.1 Source acquisition and lexical state

The first operational step is source acquisition. The driver receives a source path as its first command-line argument and optionally receives an output directory. It opens the file in binary mode, seeks to its end to determine the allocation size, allocates a zero-initialized buffer, reads the file, closes the stream, and returns the buffer to the compiler pipeline. This is a compact strategy for a classroom compiler because the later phases can use indexed access and can inspect the next character without maintaining a stream abstraction. The approach also makes error handling simple: an unavailable file produces a diagnostic and a non-zero exit status before lexical analysis begins.

Lexical scanning maintains an integer index and a one-based line counter. Newlines increment the line counter before the scanner continues. Other whitespace is skipped. Both hash comments and double-slash comments are ignored until a newline. This choice allows examples to resemble either scripting-language or C-style source. Because line information is attached to every token, later diagnostics can report the location of a malformed expression, unknown character, or missing delimiter. Line tracking is one of the smallest implementation details with the greatest debugging value: without it, a compiler report would force the user to search the entire source file for the cause.

Identifiers begin with an alphabetic character or underscore and may continue with alphanumeric characters or underscores. The scanner compares completed identifier text against the five reserved words show, if, else, true, and false. Everything else becomes an identifier. This means keyword recognition is case-sensitive, which is appropriate for a small language because it avoids silently accepting inconsistent spelling. The scanner stores token text in a bounded character array. Long names are truncated rather than dynamically expanded, an intentional implementation limit that should be documented and tested at the boundary.

Numbers accept digits and decimal points, including a leading decimal point when it is followed by a digit. A count of decimal points is used to flag values such as 12.3.4. The code still creates a number token after reporting the malformed value, but the global error flag stops later phases. This is a simple form of error containment: the lexer can continue enough to avoid an immediate crash, while the compilation decision remains failure. A more formal lexer would separate diagnostic recovery from token production and would define whether exponent notation, signed literals, or hexadecimal values are legal. Current examples establish only decimal notation.

Strings begin with a double quote and terminate at the next unescaped double quote. A backslash causes the scanner to skip one character before copying it into the token buffer. Newline before termination is an error, as is end of file before termination. The current implementation preserves the character after the backslash but does not interpret escape sequences into a richer representation. The C back end later escapes quotes and backslashes again. This is enough for the included names and grades, but a language specification should decide whether sequences such as newline, tab, and Unicode escapes are supported or rejected.

F.2 Grammar and precedence

The parser is deliberately small. A program is a statement list. A statement list can be empty, which allows an empty program to parse, or it can contain statements linked through the AST. Assignment starts with an identifier and may include an equals sign. The parser then expects an expression and a semicolon. The same path supports the result-language shorthand in which the student field can be written without an equals sign. Show accepts either parenthesized or unparenthesized expressions, although the examples mostly use parentheses. If requires a parenthesized condition, a then branch, the else keyword, and an else branch.

Expression parsing is divided into primary, multiplicative, additive, and comparison levels. Primary expressions include literals, identifiers, and parenthesized expressions. The multiplicative function consumes a primary and repeatedly attaches star or slash operators. The additive function consumes a multiplicative expression and repeatedly attaches plus or minus operators. The top expression function consumes one additive expression and optionally attaches one comparison. Consequently, `a + b * c` becomes addition whose right child is multiplication, while `a > b + c` becomes comparison whose right child is addition. The generated AST can therefore serve as a direct test of precedence rather than relying only on final numeric output.

The parser currently permits one comparison at the outer expression level rather than a chain of comparisons. That is a reasonable domain restriction because conditions normally compare one result with one threshold. It also prevents ambiguous expressions such as `a < b < c` from being silently interpreted as a sequence of Boolean and numeric operations. The Bison specification expresses the same precedence using declarations: addition and subtraction are lower than multiplication and division, and comparison is non-associative. Keeping these two descriptions aligned is important if the declarative front end is activated later.

Syntax errors set a global error flag and print a phase label, line number, expected token, and actual token where possible. The parser has limited recovery: when an invalid primary is encountered, it advances one token and creates a placeholder numeric node. The compiler does not proceed to output generation because the global error flag is checked immediately after parsing. This is adequate for rejecting a file, but it does not attempt to report multiple independent syntax errors in one run. A future diagnostic mode could synchronize at semicolons and braces, collect several errors, and still avoid producing misleading intermediate files.

F.3 AST representation

The AST is the central connection between front-end analysis and the back end. Every node has a kind and source line. Literal nodes carry their value, identifiers carry their name, binary nodes carry an operator and two children, assignments carry a variable name and value child, show carries one child, and if nodes carry condition, then, and else children. Statement lists use a first-child pointer and a next pointer on each statement. This representation avoids a separate container type while still allowing ordered traversal.

The tree is intentionally close to the source language. There is no separate desugaring phase for the optional assignment syntax, and there is no control-flow graph. This makes the AST readable in `ast.txt`. Readability is an important project outcome because students can compare the source with a structured representation and see exactly where nesting and operator precedence were resolved. The modular AST implementation also provides recursive freeing, while the compact driver uses a fixed node arena whose lifetime ends with the process.

One design detail is that type information is stored on expression nodes after semantic analysis. Before that pass, identifiers and binary expressions may be unknown. Once analysis succeeds, the AST carries enough information for both TAC and C generation to select representation and formatting. This is a lightweight form of annotation. It avoids recomputing types in the back end, but it also means code generation assumes semantic analysis has completed successfully. That assumption is enforced by the driver's error check before output files are opened.

F.4 Symbol table and static typing

The symbol table uses a linear search over a bounded array. For the small examples, this is preferable to introducing hashing because the behavior is easy to inspect. Each entry stores the name, inferred type, and first source line. A lookup returns the existing symbol or reports that an identifier is undefined. On first assignment, the inferred type is inserted. On

subsequent assignment, the existing type must match the new type. This implements a simple invariant: one variable name has one stable type throughout a compilation unit.

Type inference is driven by the right-hand expression. A numeric literal creates a number variable; a string literal creates a string variable; a comparison creates a Boolean variable. An identifier expression obtains its type from the table. This supports useful result programs without requiring declarations such as `number average;`. The cost is that a typo in a variable name is interpreted as an undefined variable rather than as a declaration error, which is a helpful failure mode for students.

Arithmetic checks both operands. If either is not numeric, a semantic diagnostic names the operator and states that number operands are required. Comparisons check that known operands have matching types. Unknown operands are allowed to avoid duplicate messages when an earlier undefined-variable error already explains the problem. An if condition must be bool. These rules are conservative enough to reject the included invalid cases while allowing string equality and Boolean equality if the operands have matching types. The language would benefit from a table that explicitly distinguishes equality from ordering and states whether string ordering is intended.

Division by a literal zero is detected by inspecting the right child after expression recursion. This catches `100 / 0` before C is generated. It does not catch `100 / mark` when mark is zero at run time. The C helper handles that case and exits with status two after writing a runtime message. The combination of static and dynamic protection is a good example of layered validation. It also illustrates why static analysis cannot be treated as a complete substitute for runtime checks in a language that permits computed values.

F.5 Three-address code and control flow

TAC gives every compound expression a named temporary. A binary expression recursively emits its left and right places, allocates the next temporary number, writes an operation line, and returns the temporary as the place representing the result. Assignments then write a simple copy from a place into a named variable. Show emits a parameter line and a call line. Although the representation is textual, it has the familiar shape of a low-level intermediate form and can be inspected without reading C syntax.

Conditional translation uses two labels. The condition is evaluated into a place, an ifFalse instruction jumps to the else label, the then list is emitted, and an unconditional jump skips to the end label. The else list is emitted between the labels. Nested conditionals allocate additional label pairs. The generated TAC for the full demonstration shows this structure clearly: each threshold comparison has an ifFalse jump, each successful branch jumps to

the final join for its level, and the nested alternatives eventually converge. This is a useful bridge between source-level if/else and machine-oriented control flow.

The current TAC generator does not perform optimization. It does not fold constant arithmetic, eliminate copies, reuse temporaries, or simplify branches. That is appropriate for the project because the purpose is visibility, not performance. An optimization phase could be added after the representation becomes a structured instruction list. Before optimization, the project should define invariants such as unique temporary names, labels that are defined exactly once, and places that are either literals, variables, or temporaries.

F.6 C generation and execution boundary

The C generator maps numbers to double, booleans to int, and strings to pointers to constant character data. It emits declarations before statements because the symbol table is available after semantic analysis. Assignments become C assignments. Numeric show expressions use two decimal places, string expressions use the string format, and Boolean expressions use a conditional expression that prints true or false. This formatting makes the result-oriented output readable and gives the language a stable presentation behavior.

Parentheses are inserted around non-division binary expressions, preserving the AST's grouping in the generated source. Division is routed through `rl_safe_div` rather than emitted directly. String literals are escaped for quotes and backslashes. The generator does not emit user-defined functions, loops, arrays, or input operations because the source language does not contain them. It emits a complete C translation unit with a main function, standard headers, declarations, statements, and return zero.

The GCC boundary is explicit in the build scripts and test runner. First, GCC compiles the driver with C11 and warnings. Then ResultLang writes generated C. A second GCC command builds that generated C and suppresses one unused-variable warning that can occur in branch-heavy examples. Finally, the executable is run and its standard output is compared exactly. This two-compiler arrangement is pedagogically useful but introduces environmental dependencies: GCC must be installed, available on PATH, and compatible with the generated C. The README correctly identifies GCC as a prerequisite.

F.7 Testing interpretation

The test runner uses exact output lists for valid examples. This is stronger than checking only an exit status because it verifies values, formatting, branch decisions, and output order. The precedence case catches an incorrect parser or generator even if both still produce compilable C. The boundary case at 80 catches an off-by-one comparison error. The nested expression and full demonstration cases exercise tree recursion and branch

lowering together. The string and Boolean case checks typed output formatting rather than only numeric printing.

The invalid tests verify three properties at once: the compiler exits unsuccessfully, the diagnostic belongs to the expected phase, and generated C is absent. The last property prevents a stale or partial output directory from being mistaken for a successful compilation. The script creates a clean test directory for each case and records diagnostics and paths in a CSV summary. This is a small but valuable reproducibility practice because a later reviewer can inspect not only the pass count but also the relevant compiler message.

Passing tests do not remove the need for negative testing beyond the ten examples. A robust extension would generate random token sequences, mutate valid programs by deleting delimiters, and vary identifier lengths and numeric forms. It would also compile generated C with sanitizers. Tests should assert that source files containing only comments, empty blocks where allowed, deeply nested conditionals, and maximum symbols behave according to an explicit specification. A separate unit suite could call lexer and semantic functions directly, while the existing pipeline suite should remain as a regression layer.

F.8 Reproducibility and maintainability

Reproducibility begins with the documented commands. A new user can run the build script, compile the full example, execute the generated program, or run the Python suite. The runner records generated artifacts under predictable directories. This structure makes the project suitable for a practical compiler-design demonstration. The missing piece is environment capture. The project should record the GCC version, Python version, operating system, and whether the run used the compact driver or a generated Flex/Bison front end.

Maintainability would improve if common data definitions were shared by the driver and modular implementation. At present, the compact driver defines its own token, type, node, and symbol structures, while the modular files define RLAstNode, RLType, and RLSymbolTable. Parallel implementations are understandable during development, but changes to syntax or diagnostics can be applied to one path and forgotten in the other. A single public AST and a single semantic API would reduce drift. The build should then compile the same modules used by the tests and examples.

Documentation should also state the intended status of every source file. The README currently explains that the verified build uses main.c because Flex and Bison were unavailable. That is good context, but a formal report should repeat it so readers do not assume parser.y is the executable parser. A short architecture note could identify the authoritative path, the experimental path, generated files, and the exact relationship

between them. This would make future team members less likely to modify a dormant component while expecting a behavior change in the running compiler.

Appendix G: Traceability Matrix

Requirement or behavior	Implementation evidence	Validation evidence	Assessment
Recognize names and keywords	Lexer rules in main.c and lexer.l	All valid examples and unknown-character rejection	Implemented for the documented vocabulary.
Support numeric, string, and Boolean values	Type enum, literal nodes, semantic expression checks	Basic assignment and string/bool examples	Implemented with intentionally small type set.
Preserve arithmetic precedence	parse_primary, parse_mul, parse_add	Precedence example and AST output	Implemented and observable.
Support conditional grade logic	If AST nodes, TAC labels, C if/else emission	Passing, failing, boundary, and full demo cases	Implemented through nested branches.
Reject undefined variables	Symbol lookup during semantic analysis	Invalid example 05	Implemented with line-numbered diagnostic.
Reject incompatible reassignment	put_sym and rl_symbol_define checks	Invalid example 06	Implemented for stable inferred types.
Reject unsafe division	Constant semantic check and rl_safe_div	Invalid example 08 and generated helper	Static and runtime protection present.
Expose intermediate artifacts	ast.txt, symbol_table.csv, intermediate.tac, generated.c	Generated sample files	Implemented and useful for assessment.

Build native executable	Batch files and Python test runner	All ten valid pipeline tests	Implemented with GCC prerequisite.
Provide interactive use	run_user_input.py and batch wrapper	Workflow documented in README	Implemented; interactive session is not automated in test_all.py.

Appendix H: Code-Level Review Notes

H.1 Driver control flow

The command-line entry point has a deliberately linear control flow. It first checks the argument count and prints a usage message if a source path is missing. It chooses the default generated directory unless the caller supplies another directory. It reads the source and immediately returns status two if reading fails. The lexical pass follows. If any lexical diagnostic has been recorded, the driver returns status one and does not allocate or write output artifacts. Parsing follows only after lexical success. The same gate appears after parsing and semantic analysis. This sequencing prevents later stages from treating partial structures as trustworthy.

After analysis succeeds, the driver constructs output paths for the AST, symbol table, TAC, and C files. It opens each file in turn and invokes the corresponding writer. The implementation assumes the output directory exists; the batch scripts create it before invoking the compiler. A more defensive driver would create the directory itself or report a clear file-opening error for each artifact. It would also check the result of every fopen call. These are small reliability improvements because a disk or permission failure should not be reported as a successful compilation.

The driver frees the source buffer before returning. The fixed arrays for tokens, nodes, and symbols are automatic process-lifetime storage and therefore do not require individual destruction. The modular AST implementation follows a different memory model and recursively frees children and sibling chains. Both models are valid for the project scale, but they should not be mixed accidentally. If the compact driver were changed to allocate nodes dynamically, ownership rules would need to be specified for placeholder nodes created during syntax recovery.

H.2 Diagnostic quality

ResultLang diagnostics are short and phase-labelled. A message such as “[semantic error] line 2: undefined variable 'mark'” gives a user enough information to locate a common

mistake. The unknown-character diagnostic quotes the offending character. The syntax expectation message reports both the expected token and the token found. The missing-else test demonstrates a structural diagnostic rather than merely a generic parse failure. These messages are appropriate for teaching because they make the compiler phase visible.

There are several ways to improve diagnostics without changing the language. The driver can preserve the original source lines and print a caret under the relevant column. Tokens can store a column in addition to a line. The parser can distinguish a missing semicolon from an invalid expression by synchronizing at a statement boundary. Semantic messages can include both the original assignment line and the later conflicting line. For a comparison mismatch, the message can name the two expressions as well as their types. Such changes would improve usability while preserving the existing error categories used by the tests.

Error policy should also be specified. Current lexical errors set a global flag, syntax errors may advance the token position, and semantic errors continue checking enough of the tree to report more than one problem in some cases. The project currently treats any error as a reason not to generate C. That is the correct default for a compiler demonstration because generated output after an error could be mistaken for a valid result. If an IDE mode were introduced, the compiler could still build a partial tree for editor feedback, but it should mark that tree as invalid and never execute it.

H.3 Portability considerations

The repository is oriented toward Windows. Batch files use backslashes, and the README assumes GCC installed through MSYS2 or MinGW. The Python test runner uses subprocess with shell execution and constructs Windows paths. The C source itself uses standard library facilities and should be portable to other operating systems, but the surrounding scripts need adaptation. A cross-platform project could provide a Makefile or CMake configuration alongside the batch files and use pathlib-based subprocess argument lists rather than shell strings.

Portability also includes compiler behavior. The source uses C11-compatible constructs and is compiled with warnings enabled. Numeric output depends on the formatting rule selected by the generator, which is stable at two decimal places for number output. Floating-point behavior for calculations such as averages is inherited from double arithmetic. The language specification should state that numbers are real-valued for presentation purposes and should document rounding expectations. If exact marks or currency values were required, a decimal or integer representation would be more appropriate.

Generated C assumes that source identifiers are legal C identifiers. ResultLang identifiers use a similar initial-character and continuation rule, but the language does not appear to reserve C keywords such as int, if, or return. A source program using such a name could generate invalid or surprising C. The safest remedy is name mangling: every source identifier can be mapped to a generated prefix plus an encoded source name, while the symbol table retains the original spelling for diagnostics. A simpler remedy is to reserve target keywords during semantic analysis and report them before generation.

H.4 Security and robustness perspective

This project does not process network data or credentials, so its primary risks are robustness and unsafe generated code rather than web security. It still benefits from secure coding principles. Bounded buffers reduce some overflow risk, but truncation can produce two different long identifiers with the same stored prefix. That could make one source variable appear to be another. The implementation should either reject names longer than the limit or allocate exact storage. The same consideration applies to string literals and token text.

The test runner constructs shell command strings containing paths. In the controlled repository examples this is acceptable, but a general tool should pass argument arrays without shell interpretation. Source paths containing quotes or shell metacharacters could otherwise alter the command. The generated C compiler invocation should likewise avoid string concatenation when a subprocess API can pass separate arguments. These observations do not undermine the current test result; they define the boundary between a controlled classroom runner and a reusable compiler service.

Generated code should never be executed if semantic validation fails. The current driver enforces that rule by returning before writing generated.c. The test suite explicitly checks the absence of that file for invalid cases. This is a strong design decision. If generated outputs are retained from a previous run, the scripts should clean or isolate the output directory so that a stale executable cannot be confused with the current source. The test runner's per-case directories already provide that isolation.

H.5 Educational value

ResultLang is especially effective as a teaching artifact because it exposes intermediate forms that many introductory examples omit. A student can write a short grade program, inspect the parse tree, locate inferred types, follow temporary values in TAC, read the generated C, and run the executable. This sequence makes abstract compiler terminology concrete. The full demonstration is also domain-motivated: a total, average, and grade threshold are easier to understand than an arbitrary expression with no visible purpose.

The project shows that a compiler is a chain of contracts. The lexer promises that characters become meaningful tokens. The parser promises that tokens become a structured program. Semantic analysis promises that the structure obeys type and name rules. TAC promises that evaluation and control flow can be represented explicitly. C generation promises that the analyzed program can be expressed in a target language. GCC and the executable provide the final observable result. A failure at one contract should stop dependent contracts from operating, which is exactly how the driver's phase gates are arranged.

The project also creates opportunities for guided exercises. Students can add a modulus operator and follow it through the lexer, grammar, semantic rules, TAC, C generator, examples, and tests. They can add a unary minus and observe how precedence changes. They can replace linear symbol lookup with hashing and measure the conceptual trade-off. They can add a warning for variables assigned but never shown. Each extension reinforces the principle that a language feature is a cross-pipeline change rather than a local edit.

H.6 Suggested marking criteria

A fair evaluation should distinguish implementation completeness from language breadth. Completeness can be assessed through successful builds, correct artifacts, valid output, and rejection of invalid programs. Front-end quality can be assessed through token coverage, precedence, source locations, and error messages. Semantic quality can be assessed through undefined-name checks, stable inferred types, operator rules, and division safety. Back-end quality can be assessed through valid generated C, correct typed output, and control-flow preservation. Testing quality can be assessed through positive and negative cases, exact expectations, clean output directories, and reproducible commands.

The current project performs strongly on the complete tested pipeline. It should not be penalized for lacking loops, functions, arrays, or user input because the README explicitly defines those as limitations. It should, however, receive constructive feedback for the split between the verified driver and modular source files. A high-quality submission should make it obvious which implementation is built and should avoid leaving a reader to infer whether parser.y is live. This is a maintainability concern rather than a failure of the demonstrated language behavior.

Appendix I: Example Diagnostic Catalogue

Input condition	Expected phase	Meaning	Recommended user action

Character such as @ appears outside a string	Lexical	The character has no token rule.	Remove it or use a supported operator.
String reaches newline without closing quote	Lexical	The literal is incomplete.	Add the closing quote on the same line.
Number contains multiple decimal points	Lexical	The literal is not a supported decimal.	Use one decimal point.
Statement lacks semicolon	Syntax	The statement boundary is incomplete.	Terminate the statement with semicolon.
Expression begins with an operator	Syntax	No valid primary expression was found.	Provide a literal, identifier, or parenthesized expression.
If has no else branch	Syntax	The language requires both branches.	Add else and its statement or block.
Identifier is used before assignment	Semantic	The symbol table has no matching entry.	Assign the variable before using it.
Variable changes from string to number	Semantic	Inferred type is stable after first assignment.	Use a new variable name or preserve the original type.
String is added to a number	Semantic	Arithmetic is numeric-only.	Use numeric operands for arithmetic.
Comparison combines different types	Semantic	Operands do not have compatible types.	Compare values of the same type.
Literal divisor is zero	Semantic	The compiler can prove the operation is unsafe.	Change the divisor or handle the value before division.

Appendix J: Final Assessment Statement

On the evidence inspected, ResultLang meets the core objectives of a small compiler-design project. It has a defined domain, an executable implementation, explicit intermediate artifacts, a target-language back end, reproducible build commands, and a balanced set of valid and invalid tests. The strongest conclusion is not that the project implements every compiler feature; it is that the selected features form a complete and testable vertical slice. A reader can move from source characters to a native executable and can observe the decisions made at each stage.

The project is ready for academic submission after the report is formatted and the generated test summaries are included as supporting evidence. For a future engineering release, the first priority should be to unify the authoritative implementation and expand robustness testing. The current results support confidence in the ten valid examples and ten invalid examples, while the limitations section correctly prevents overclaiming. This balance between demonstrated success and stated boundary is an important part of a credible technical report.

Appendix K: Implementation Glossary and Reproducibility Protocol

K.1 Glossary

AST: An abstract syntax tree is the nested representation of a program after irrelevant punctuation has been removed and meaningful relationships have been preserved. In ResultLang, an AST records programs, lists, assignments, output statements, conditionals, literals, identifiers, and binary operations. The AST is abstract because it does not retain every source character, but it retains enough structure for semantic checking and code generation.

Token: A token is a classified piece of source text such as an identifier, number, keyword, operator, or delimiter. The lexer converts a character stream into tokens so the parser does not need to reason about individual characters. ResultLang tokens also retain line information and, for numbers and strings, a value or text payload.

Lexeme: A lexeme is the actual source spelling that produced a token. The identifier lexeme `average` becomes an identifier token whose text is average. Retaining lexemes allows diagnostics to quote names and allows later stages to refer to symbols without returning to the original source buffer.

Grammar: A grammar defines which token sequences form valid programs. ResultLang's grammar includes statement lists, assignments, show statements, conditionals, blocks, and expressions. The recursive-descent functions in the verified driver and the productions in

parser.y are two descriptions of this grammar, although only the former is used by the tested build.

Precedence: Precedence determines which operators bind more tightly. ResultLang gives multiplication and division higher precedence than addition and subtraction, and gives comparisons lower precedence than arithmetic. This means a total can be written naturally without excessive parentheses. The precedence test demonstrates that the chosen rules affect the final numerical output.

Semantic analysis: Semantic analysis checks properties that cannot be decided from token order alone. It verifies that names exist, operands have compatible types, conditions are Boolean, assignments preserve inferred types, and statically visible zero division is rejected. The result of this phase is an annotated tree and a populated symbol table.

Symbol table: A symbol table maps source names to information needed by later phases. ResultLang stores the inferred type and first assignment line. The table is used during identifier lookup, reassignment checking, declaration generation, and artifact inspection. The CSV dump makes the table visible to a reviewer.

Intermediate representation: An intermediate representation is a form between the source language and the target language. ResultLang's textual TAC uses temporaries, variable copies, parameters, calls, conditional jumps, and labels. It is intentionally simple and is designed to expose evaluation and control flow rather than to provide an optimized compiler IR.

Code generation: Code generation maps analyzed structures to a target language. ResultLang emits C declarations, expressions, assignments, output calls, and if/else statements. It also emits a helper for runtime division safety. The target is selected for availability and readability rather than for a custom virtual machine.

Pipeline test: A pipeline test validates several compiler stages together. In this project, a valid case is accepted by the ResultLang driver, translated to C, compiled by GCC, executed, and compared with expected lines. An invalid case is expected to stop before generated C is produced. This test style provides stronger integration evidence than checking parser return values alone.

K.2 Reproducibility protocol

A clean evaluation should begin from the repository root. The evaluator confirms that GCC is available by running its version command and confirms that Python is available for the test harness. The evaluator then runs the build batch file or its equivalent command, checks that the compiler executable exists, and compiles the full demonstration with the file

compilation script. The generated directory should contain the AST, symbol table, TAC, generated C, and executable after a successful run.

The evaluator next runs the Python suite. The suite creates an outputs directory, rebuilds the compiler, removes any previous per-test directory, and processes the ten valid examples in sorted order. For each valid case, the evaluator should inspect the status, the expected output comparison, and the generated C compilation status. The run is complete only when the executable also returns zero. The final transcript should report the number of tests and failures rather than only printing a successful word.

Invalid cases should be reviewed separately from valid cases. The evaluator should confirm that an unknown character stops during lexing, a missing delimiter or malformed expression stops during parsing, and undefined names, type mismatches, invalid operations, and zero division stop during semantic analysis. The relevant diagnostic text should include the expected phase. The output directory for each invalid case should not contain generated C. This final check protects against accidental execution of stale artifacts.

For a formal submission, the evaluator should archive the test summary CSV, Markdown summary, transcript, representative AST, symbol table, TAC, and generated C. The archive should include the date, platform, GCC version, Python version, and command lines. If the generated files are overwritten by a later example, the full-result example should be copied into a named evidence directory before another compilation is run. That preserves a stable relation between the source program and each reported artifact.

A second reproducibility pass should intentionally test a changed example. The evaluator can alter a threshold from 80 to 90, rerun the compiler, and confirm that the branch result changes while the parser, semantic checks, and target compilation still succeed. A third pass can introduce an undefined name and confirm that the compiler returns a failure status and produces no new generated C. These small perturbation checks establish that the pipeline responds to source changes rather than merely replaying a cached executable.

Finally, a reviewer should compare the report's claims with the source and artifacts. Claims about supported syntax should appear in the lexer, parser, or examples. Claims about type checking should appear in semantic code and invalid tests. Claims about code generation should appear in generated C. Claims about test success should match the transcript. This evidence discipline prevents a polished report from overstating behavior that exists only in a planned or unused module.

Appendix L: Closing Evaluation Notes

The most appropriate way to understand ResultLang is as a deliberately narrow language with a complete vertical implementation. Its narrowness is a strength because it keeps the relationship between source constructs and generated behavior visible. A number, a string, a comparison, or a branch can be followed through scanning, parsing, annotation, intermediate lowering, target emission, and execution. The project therefore gives more useful evidence about compiler integration than a larger but unfinished language would provide.

The evidence also supports a careful distinction between correctness and completeness. Correctness means that the implemented and tested cases behave as intended. The 20 passing tests support that conclusion for the documented examples. Completeness would mean that every desired language feature, error recovery policy, capacity boundary, and target-language concern had been specified and validated. ResultLang does not claim that broader status, and the report records the missing features and risks. That restraint improves the credibility of the assessment.

For a final academic presentation, the strongest demonstration is to show the full-result source beside the generated AST, symbol table, TAC, and C. The presenter can then run the executable and change one mark or threshold to demonstrate a different branch. A second demonstration should show an invalid program and its phase-labelled diagnostic. Together, these examples communicate both the successful path and the compiler's responsibility to reject unsafe or meaningless input. They also make the report's central conclusion directly observable.

The project provides a sound foundation for future language work. New features can be added incrementally if each one is traced through the same stages and accompanied by positive and negative tests. The recommended architectural improvement is to make the modular components and the deployed driver one implementation. Once that is done, later enhancements such as richer diagnostics, explicit declarations, loops, functions, or a structured IR can be evaluated without uncertainty about which source files control behavior. The current submission is therefore both a finished demonstration and a clear starting point for a more maintainable compiler.

In summary, the implementation, generated artifacts, automated suite, and documentation form a consistent assessment package. The report records both the successful observed behavior and the boundaries that remain. That combination gives an examiner enough detail to reproduce the result, inspect the compiler stages, and judge the design decisions independently. It also gives the development team a practical sequence for improvement: unify the front end, strengthen diagnostics, test limits, harden generation, and preserve the transparent artifact workflow that makes ResultLang easy to study.

This final evidence package connects implementation claims to source files, generated outputs, tests, and reproducible commands for independent academic review.

It is ready for submission, printing, discussion, and future maintenance by the project team and course examiner.